

Nama : Yudhistira Putra Hartanto

Kelas/Absen : 1G/29

NIM : 254107020083

Praktikum 1: Operasi Dasar Queue

- Class Queue29

```
package P1Jobsheet10;
```

```
public class Queue29 {  
    int[] data;  
    int front;  
    int rear;  
    int size;  
    int max;
```

```
    public Queue29(int n){  
        max = n;  
        data = new int[max];  
        size = 0;  
        front = rear = -1;  
    }
```

```
    public boolean IsEmpty() {  
        if (size == 0) {  
            return true;  
        } else {  
            return false;  
        }  
    }
```

```
    public boolean IsFull() {  
        if (size == max) {  
            return true;  
        } else {  
            return false;  
        }  
    }
```

```
    public void peek() {  
        if (!IsEmpty()) {  
            System.out.println("Elemen terdepan: " + data[front]);  
        } else {  
            System.out.println("Queue masih kosong");  
        }  
    }
```

```
    public void print() {  
        if(IsEmpty()) {  
            System.out.println("Queue masih kosong");  
        }
```

```

    } else {
        int i = front;
        while (i != rear) {
            System.out.println(data[i] + " ");
            i = (i + 1) % max;
        }
        System.out.println(data[i] + " ");
        System.out.println("Jumlah elemen = " + size);
    }
}

public void clear() {
    if (!IsEmpty()) {
        front = rear = -1;
        size = 0;
        System.out.println("Queue berhasil dikosongkan");
    } else {
        System.out.println("Queue masih kosong");
    }
}

public void Enqueue(int dt) {
    if (IsFull()) {
        System.out.println("Queue sudah penuh");
    } else {
        if (IsEmpty()) {
            front = rear = 0;
        } else {
            if (rear == max - 1) {
                rear = 0;
            } else {
                rear++;
            }
        }
        data[rear] = dt;
        size++;
    }
}

public int Dequeue() {
    int dt = 0;
    if (IsEmpty()) {
        System.out.println("Queue masih kosong");
    } else {
        dt = data[front];
        size--;
        if (IsEmpty()) {
            front = rear = -1;
        } else {
            if (front == max - 1) {
                front = 0;
            } else {
                front++;
            }
        }
    }
}

```

```

    }
    }
}
return dt;
}
}

```

- Class QueueMain29

```
package P1Jobsheet10;
```

```
import java.util.Scanner;
```

```

public class QueueMain29 {
    public static void menu() {
        System.out.println("Masukkan Operasi yang Diinginkan");
        System.out.println("1. Enqueue");
        System.out.println("2. Dequeue");
        System.out.println("3. Print");
        System.out.println("4. Peek");
        System.out.println("5. Clear");
        System.out.println("-----");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner (System.in);
        int pilih;
        System.out.print("Masukkan Kapasitas queue : ");
        int n = sc.nextInt();

        Queue29 Q = new Queue29(n);

        do {
            menu();
            System.out.print("Masukkan Menu :");
            pilih = sc.nextInt();
            switch (pilih) {
                case 1:
                    System.out.print("Masukkan data baru: ");
                    int dataMasuk = sc.nextInt();
                    Q.Enqueue(dataMasuk);
                    break;
                case 2:
                    int dataKeluar = Q.Dequeue();
                    if (dataKeluar != 0) {
                        System.out.println("Data yang dikeluarkan: " + dataKeluar);
                        break;
                    }
                case 3:
                    Q.print();
                    break;
                case 4:

```

```

        Q.peek();
        break;
    case 5:
        Q.clear();
        break;
    }
} while (pilih == 1 || pilih == 2 || pilih == 3 || pilih == 4 || pilih == 5);
}
}

```

- Hasil

Masukkan Kapasitas queue : 3

Masukkan Operasi yang Diinginkan

1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear

Masukkan Menu :1

Masukkan data baru: 2

Masukkan Operasi yang Diinginkan

1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear

Masukkan Menu :1

Masukkan data baru: 5

Masukkan Operasi yang Diinginkan

1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear

Masukkan Menu :1

Masukkan data baru: 4

Masukkan Operasi yang Diinginkan

1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear

Masukkan Menu :1

Masukkan data baru: 1

Queue sudah penuh

Masukkan Operasi yang Diinginkan

1. Enqueue
2. Dequeue

3. Print
4. Peek
5. Clear

Masukkan Menu :3

2
5
4

Jumlah elemen = 3

Masukkan Operasi yang Diinginkan

1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear

Masukkan Menu :

- **Pertanyaan**

1. Pada konstruktor, mengapa nilai awal atribut front dan rear bernilai -1, sementara atribut size bernilai 0?
2. Pada method Enqueue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (rear == max - 1) {
    rear = 0;
```

3. Pada method Dequeue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (front == max - 1) {
    front = 0;
```

4. Pada method print, mengapa pada proses perulangan variabel i tidak dimulai dari 0 (int i=0), melainkan int i=front?
5. Perhatikan kembali method print, jelaskan maksud dari potongan kode berikut!

```
i = (i + 1) % max;
```

- 6.
7. Tunjukkan potongan kode program yang merupakan queue overflow!
8. Pada saat terjadi queue overflow dan queue underflow, program tersebut tetap dapat berjalan dan hanya menampilkan teks informasi. Lakukan modifikasi program sehingga pada saat terjadi queue overflow dan queue underflow, program dihentikan!

Jawaban :

1. Karena indeks array berawal dari 0 jadi itu ada dibawah 1 makanya front dan rear mulainya dari -1 yang menandakan array tersebut masih kosong. Sedangkan size = 0 itu menunjukkan jumlah elemen dalam queue adalah nol atau masih kosong.
2. Kode ini memiliki kegunaan dimana untuk mengecek kondisi jika rear atau kode terakhir dimasukkan apakah sudah sama dengan indeks terakhir array (max-1), jika sama maka rear akan direset kembali ke 0.

3. Untuk mengecek kondisi ketika front sama dengan indeks terakhir array, maka setelah dilakukan penghapusan, maka front akan dipindahkan ke indeks 0
4. Karena dalam queue, elemen yang pertama kali masuk (paling depan) berada pada indeks front, bukan pada indeks 0
5. Jika $i + 1$ kurang dari max, hasilnya adalah $i + 1$ (bergeser ke indeks berikutnya) dan jika $i + 1$ sama dengan max, hasilnya adalah 0 (kembali ke awal). Ini untuk memungkinkan perulangan mencetak queue dengan benar meskipun front lebih besar dari rear.
6.

```
if (IsFull()) {
    System.out.println("Queue sudah penuh");
}
```
7. Perbaikan :

```
do {
    menu();
    System.out.print("Masukkan Menu :");
    pilih = sc.nextInt();
    switch (pilih) {
        case 1:
            if(Q.IsFull()){
                System.out.print("penuh");
                pilih = 6;
                break;
            }else{
                System.out.print("Masukkan data baru: ");
                int dataMasuk = sc.nextInt();
                Q.Enqueue(dataMasuk);
                break;
            }

        case 2:
            if(Q.IsEmpty()){
                System.out.print("penuh");
                pilih = 6;
                break;
            }else{
                int dataKeluar = Q.Dequeue();
                if (dataKeluar != 0) {
                    System.out.println("Data yang dikeluarkan: " + dataKeluar);
                    break;
                }
            }
    }
}
```

Hasil :

Masukkan Kapasitas queue : 3

Masukkan Operasi yang Diinginkan

1. Enqueue

2. Dequeue

3. Print

4. Peek

5. Clear

Masukkan Menu :1

Masukkan data baru: 2

Masukkan Operasi yang Diinginkan

1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear

Masukkan Menu :1

Masukkan data baru: 3

Masukkan Operasi yang Diinginkan

1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear

Masukkan Menu :1

Masukkan data baru: 4

Masukkan Operasi yang Diinginkan

1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear

Masukkan Menu :1

penuh

PS D:.\Kuliah\Semester 2\Materi & Tugas

PASD\PraktikumAlgoritmaStrukturData>

Praktikum 2 : Antrian Layanan Akademik

- Class Mahasiswa29

package P2Jobsheet10;

```
public class Mahasiswa29 {
    String nim;
    String nama;
    String prodi;
    String kelas;

    public Mahasiswa29(String nim, String nama, String prodi, String kelas){
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;
        this.kelas = kelas;
    }

    public void tampilkandata(){
        System.out.println(nim + " - " + nama + " - " + prodi + " - " + kelas);
    }

}
```

- Class AntrianLayanan29

package P2Jobsheet10;

```
public class AntrianLayanan29 {
    Mahasiswa29[] data;
    int front;
    int rear;
    int size;
    int max;

    public AntrianLayanan29(int max){
        this.max = max;
        this.data = new Mahasiswa29[max];
        this.front = 0;
        this.rear = -1;
        this.size = 0;
    }

    public boolean IsEmpty() {
        if (size == 0) {
            return true;
        } else {
            return false;
        }
    }

    public boolean IsFull() {
        if (size == max) {
```



```

        return true;
    } else {
        return false;
    }
}

public void lihatTerdepan() {
    if (IsEmpty()) {
        System.out.println("Antrian kosong.");
    } else {
        System.out.print("Mahasiswa terdepan: ");
        System.out.println("NIM - NAMA - PRODI - KELAS");
        data[front].tampilKandata();
    }
}

public void tampilkanSemua() {
    if (IsEmpty()) {
        System.out.println("Antrian kosong.");
        return;
    }
    System.out.println("Daftar Mahasiswa dalam Antrian:");
    System.out.println("NIM - NAMA - PRODI - KELAS");
    for (int i = 0; i < size; i++) {
        int index = (front + i) % max;
        System.out.print((i + 1) + ". ");
        data[index].tampilKandata();
    }
}

public void clear() {
    if (!IsEmpty()) {
        front = rear = -1;
        size = 0;
        System.out.println("Queue berhasil dikosongkan");
    } else {
        System.out.println("Queue masih kosong");
    }
}

public void tambahAntrian(Mahasiswa29 mhs) {
    if (IsFull()) {
        System.out.println("Antrian penuh, tidak dapat menambah mahasiswa.");
        return;
    }
    rear = (rear + 1) % max;
    data[rear] = mhs;
    size++;
    System.out.println(mhs.nama + " berhasil masuk ke antrian.");
}

public Mahasiswa29 layaniMahasiswa() {
    if (IsEmpty()) {

```

```

        System.out.println("Antrian kosong.");
        return null;
    }
    Mahasiswa29 mhs = data[front];
    front = (front + 1) % max;
    size--;
    return mhs;
}

public int getJumlahAntrian() {
    return size;
}

}

```

- **Class LayananAkademikSIKAD**

```
package P2Jobsheet10;
```

```
import java.util.Scanner;
```

```

public class LayananAkademikSIKAD29 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianLayanan29 antrian = new AntrianLayanan29(5);
        int pilihan;

        do {
            System.out.println("\n=== Menu Antrian Layanan Akademik ===");
            System.out.println("1. Tambah Mahasiswa ke Antrian");
            System.out.println("2. Layani Mahasiswa");
            System.out.println("3. Lihat Mahasiswa Terdepan");
            System.out.println("4. Lihat Semua Antrian");
            System.out.println("5. Jumlah Mahasiswa dalam Antrian");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");
            pilihan = sc.nextInt();
            sc.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("NIM : ");
                    String nim = sc.nextLine();
                    System.out.print("Nama : ");
                    String nama = sc.nextLine();
                    System.out.print("Prodi : ");
                    String prodi = sc.nextLine();
                    System.out.print("Kelas : ");
                    String kelas = sc.nextLine();
                    Mahasiswa29 mhs = new Mahasiswa29(nim, nama, prodi, kelas);
                    antrian.tambahAntrian(mhs);
                    break;

```

```

        case 2:
            Mahasiswa29 dilayani = antrian.layaniMahasiswa();
            if ( dilayani != null) {
                System.out.println("Mahasiswa yang dilayani:");
                dilayani.tampilKandata();
            }
            break;

        case 3:
            antrian.lihatTerdepan();
            break;

        case 4:
            antrian.tampilkanSemua();
            break;

        case 5:
            System.out.println("Jumlah mahasiswa dalam antrian: " +
antrian.getJumlahAntrian());
            break;

        case 0:
            System.out.println("Terima kasih.");
            break;

        default:
            System.out.println("Pilihan tidak valid!");
    }
} while (pilihan != 0);

sc.close();

}
}

```

- Hasil

=== Menu Antrian Layanan Akademik ===

1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar

Pilih menu: 1

NIM : 123

Nama : Aldi

Prodi : TI

Kelas : 1A

Aldi berhasil masuk ke antrian.

=== Menu Antrian Layanan Akademik ===

1. Tambah Mahasiswa ke Antrian

2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 1
NIM : 124
Nama : Bobi
Prodi : TI
Kelas : 1G
Bobi berhasil masuk ke antrian.

==== Menu Antrian Layanan Akademik ====

1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 4
Daftar Mahasiswa dalam Antrian:
NIM - NAMA - PRODI - KELAS
1. 123 - Aldi - TI - 1A
2. 124 - Bobi - TI - 1G

==== Menu Antrian Layanan Akademik ====

1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 2
Mahasiswa yang dilayani:
123 - Aldi - TI - 1A

==== Menu Antrian Layanan Akademik ====

1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 4
Daftar Mahasiswa dalam Antrian:
NIM - NAMA - PRODI - KELAS
1. 124 - Bobi - TI - 1G

==== Menu Antrian Layanan Akademik ====

1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian

5. Jumlah Mahasiswa dalam Antrian

0. Keluar

Pilih menu: 0

Terima kasih.

- **Pertanyaan**

Lakukan modifikasi program dengan menambahkan method baru bernama LihatAkhir pada class AntrianLayanan yang digunakan untuk mengecek antrian yang berada di posisi belakang. Tambahkan pula daftar menu 6. Cek Antrian paling belakang pada class LayananAkademikSIAKAD sehingga method LihatAkhir dapat dipanggil!

Jawaban :

- Class AntrianLayanan29

```
public void LihatAkhir() {
    if (IsEmpty()) {
        System.out.println("Antrian kosong.");
    } else {
        System.out.println("Mahasiswa paling belakang: ");
        System.out.println("NIM - NAMA - PRODI - KELAS");
        data[rear].tampilKandata();
    }
}
```
- Class LayananAkademikSIAKAD29

```
case 6:
    antrian.LihatAkhir();
    break;
```
- Hasil

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
6. Lihat Mahasiswa Paling Belakang
0. Keluar
Pilih menu: 1
NIM : 123
Nama : aldi
Prodi : TI
Kelas : 1A
aldie berhasil masuk ke antrian.

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
6. Lihat Mahasiswa Paling Belakang
```

0. Keluar
Pilih menu: 1
NIM : 124
Nama : bobi
Prodi : TI
Kelas : 1G
bobi berhasil masuk ke antrian.

==== Menu Antrian Layanan Akademik ====

1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
6. Lihat Mahasiswa Paling Belakang

0. Keluar
Pilih menu: 1
NIM : 125
Nama : yudhis
Prodi : 1H
Kelas : TI
yudhis berhasil masuk ke antrian.

==== Menu Antrian Layanan Akademik ====

1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
6. Lihat Mahasiswa Paling Belakang

0. Keluar
Pilih menu: 6
Mahasiswa paling belakang:
NIM - NAMA - PRODI - KELAS
125 - yudhis - 1H - TI